# Progressive Web App

PWA jest to skrót od wyrażenia **Progressive Web App (Progresywna Aplikacja Internetowa).** Jest to zwykła aplikacja internetowa (HTML, CSS, JavaScript), która dzięki kilku dodatkom — **manifestowi** i **service workerowi** — może zostać zainstalowana na urządzeniu jak aplikacja natywna i działać także bez dostępu do Internetu. Przeglądarka sama z siebie nie zapisuje strony do pracy offline — to service worker decyduje, które zasoby trafią do pamięci podręcznej (cache) i skąd będą pobierane przy kolejnych wizytach.

Cechy charakterystyczne aplikacji PWA:

- **Responsywność** - wygląd strony powinien dopasować się do każdego ekranu,
- **Szybkość** - strony PWA powinny ładować się szybko nawet przy słabym łączu internetowym. Wydajność może grać kluczową rolę w utrzymaniu użytkowników — chętniej korzystają oni z witryn, które działają szybko, niż z tych, które działają powoli,
- **Progresywne ulepszanie (progressive enhancement)** - aplikacja działa w każdej przeglądarce, a dodatkowe możliwości (instalacja, praca offline, powiadomienia) włączają się tam, gdzie przeglądarka je obsługuje,
- **Niestandardowa strona offline** - w przypadku braku połączenia internetowego aplikacja PWA wyświetla własną stronę zamiast domyślnego komunikatu przeglądarki o braku sieci,
- **Instalowalność** - aplikacja PWA może zostać zainstalowana i uruchamiana z ekranu głównego / menu systemu, w osobnym oknie.

Dodatkowo, aby stworzyć aplikację najwyższej jakości, możemy także zapewnić:

- **pełną pracę w trybie offline** - w przypadku gdy połączenie nie jest wymagane, aplikacja działa tak samo w trybie offline jak i online,
- **pełną dostępność** - według Web Content Accessibility Guidelines (WCAG 2.2),
- **możliwość znalezienia przez wyszukiwarkę**,
- **działanie z dowolnym typem wejścia** - myszka, klawiatura, dotyk itp.,
- **postępowanie zgodnie z najlepszymi praktykami dotyczącymi dobrego kodu**,
- **kontekst dla próśb o pozwolenie** na korzystanie z zaawansowanych API (np. powiadomień, geolokalizacji) — pytamy wtedy, gdy użytkownik wie, po co dana funkcja jest potrzebna.

# Jak stworzyć pierwszą aplikację typu PWA?

Sama aplikacja PWA nie wymaga Node.js — to zwykłe pliki statyczne. Node.js (a dokładniej `npx` i `npm`) jest nam potrzebny do uruchomienia narzędzi pomocniczych: lokalnego serwera, generatora ikon i Firebase CLI. Potrzebna jest wersja **LTS**, co najmniej 22.12 (tyle wymaga generator ikon).

W pracowni Node.js jest już zainstalowany — sprawdzamy tylko jego wersję:

```
node --version
```

Praca w domu: jeśli polecenie nie działa albo wersja jest starsza, instalujemy aktualną wersję LTS ze strony https://nodejs.org/en/download/ (po instalacji trzeba otworzyć terminal na nowo).

W kolejnym kroku tworzymy folder na laboratorium (np. `lab_01`), a w nim podfolder `public` — tam będzie nasza aplikacja (taki układ przyda się przy publikacji na Firebase). Wszystkie polecenia `npx` z tej części wykonujemy w folderze `public`. Tworzymy w nim 4 pliki:

- `index.html`,
- `index.js`,
- `service-worker.js`,
- `manifest.json`,

oraz (później) `offline.html`. Dodatkowo wrzucamy do folderu grafikę `android.png`.

W pliku `index.html` umieszczamy przykładowy szkic strony:

```
<!DOCTYPE html>
<html lang="pl">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#000000" />
    <title>PWA Example</title>
    <link rel="manifest" href="manifest.json" />
    <script src="index.js" defer></script>
  </head>
  <body>
    <h1>hello world</h1>
    <img src="android.png" alt="Logo Androida" width="210" />
  </body>
</html>
```

W pliku `index.js` umieszczamy skrypt rejestrujący service workera:

```
if ('serviceWorker' in navigator) {
  navigator.serviceWorker
    .register('./service-worker.js')
    .then((reg) => console.log('Service worker zarejestrowany, zakres:', reg.scope))
    .catch((err) => console.error('Błąd rejestracji service workera:', err));
}
```

Skrypt ten pozwala nam na zarejestrowanie service workera. Jeśli przeglądarka go obsługuje, jest on rejestrowany w witrynie przy użyciu metody `ServiceWorkerContainer.register()`. Jego kod znajduje się w pliku `service-worker.js` i zostaje wykonany po pomyślnej rejestracji.

Uwaga: ścieżka `./service-worker.js` jest **względna**. Gdybyśmy napisali `/service-worker.js`, przeglądarka szukałaby pliku w głównym katalogu domeny — to przestaje działać, gdy aplikacja jest opublikowana w podkatalogu (np. na GitHub Pages pod adresem `https://login.github.io/repozytorium/`).

W manifeście powinniśmy umieścić podstawowe informacje na temat naszej aplikacji:

```
{
  "id": "./",
  "name": "Moja pierwsza PWA",
  "short_name": "MojaPWA",
  "start_url": "./",
  "scope": "./",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#000000",
  "orientation": "portrait-primary",
  "icons": []
}
```

Następujące pola służą do:

- **id** - jednoznacznej identyfikacji aplikacji przez przeglądarkę (gdy go brak, używany jest `start_url`),
- **name** - pełnej nazwy aplikacji, wyświetlanej m.in. w oknie instalacji i na ekranie startowym,
- **short_name** - skróconej nazwy wyświetlanej pod ikoną aplikacji, gdy brakuje miejsca na pełną,
- **start_url** - adresu strony otwieranej po uruchomieniu zainstalowanej aplikacji,
- **scope** - zakresu adresów, które „należą" do aplikacji (poza nim otworzy się zwykła przeglądarka),
- **display** - określenia sposobu wyświetlania aplikacji. `standalone` - jest to tryb, który ukrywa interfejs przeglądarki (pasek adresu, przyciski), więc aplikacja wygląda jak natywna,
- **background_color** - koloru tła ekranu startowego (splash screen) wyświetlanego podczas uruchamiania,
- **theme_color** - koloru interfejsu systemu wokół aplikacji (np. paska stanu / paska tytułu okna),
- **orientation** - określenia orientacji wyświetlanej aplikacji (portrait, landscape itp.),
- **icons** - ikon aplikacji w różnych rozmiarach (ekran główny, menu systemu, ekran startowy). Do instalacji wymagane są co najmniej ikony **192×192** i **512×512** px.

Tablica `icons` jest na razie pusta — **bez ikon aplikacji nie da się zainstalować**, więc w następnym kroku je wygenerujemy.

Do generowania ikon możemy skorzystać z gotowych rozwiązań dostępnych w Internecie lub zrobić to ręcznie. My skorzystamy z narzędzia `pwa-asset-generator`, które na podstawie jednego obrazka (`android.png`) tworzy komplet ikon w wymaganych rozmiarach i od razu dopisuje je do manifestu. Otwieramy terminal w folderze projektu i wpisujemy:

```
npx pwa-asset-generator android.png icons --icon-only --favicon --type png --background
"#ffffff" --manifest manifest.json --index index.html
```

Znaczenie opcji:

- `--icon-only` - generuje tylko ikony (bez kilkudziesięciu ekranów startowych dla iOS),
- `--favicon` - dodatkowo generuje favicon i dopisuje go do `index.html`,
- `--type png` - zapisuje ikony jako PNG,
- `--background "#ffffff"` - wyśrodkowuje obrazek na białym, kwadratowym tle (nasz `android.png` nie jest kwadratowy),
- `--manifest manifest.json` - automatycznie uzupełnia tablicę `icons` w manifeście,

- `--index index.html` - automatycznie dopisuje znaczniki ikon (m.in. `apple-touch-icon` dla iOS) do `index.html`.

Przy pierwszym uruchomieniu `npx` zapyta o zgodę na pobranie pakietu (`Need to install the following packages: pwa-asset-generator ... Ok to proceed? (y)`) — potwierdzamy klawiszem `y`. Narzędzie może też pobrać przeglądarkę Chromium (służy mu do renderowania ikon), więc chwilę to potrwa. Po wykonaniu komendy powinien pojawić się folder `icons`, a manifest i `index.html` powinny zostać uzupełnione — **sprawdźmy to**, otwierając oba pliki. Generator przy okazji lekko przeformatowuje `index.html` (np. `<meta ... />` zamienia na `<meta ...>`, a `defer` na `defer=""`) — to nie błąd, oba zapisy są poprawne. Każda ikona jest wpisana do manifestu dwukrotnie: z `"purpose": "any"` (zwykła ikona) oraz `"purpose": "maskable"` — ta druga może zostać przycięta przez system (np. Android) do kształtu koła lub zaokrąglonego kwadratu, dlatego obrazek ma margines (padding).

Generator dopisuje do `index.html` także znacznik `<meta name="apple-mobile-web-app-capable" ...>`, na który Chrome w konsoli zgłasza ostrzeżenie o przestarzałej nazwie. Możemy dodać obok niego nowszą wersję:

```
<meta name="mobile-web-app-capable" content="yes" />
```

Do naszego pliku `service-worker.js` wklejamy następujący kod:

```
importScripts('https://storage.googleapis.com/workbox-cdn/releases/7.4.1/workbox-sw.js');

const { precaching, routing, strategies } = workbox;

// 1. Pliki "szkieletu" aplikacji zapisujemy w cache od razu przy instalacji service
workera.
//    Zmiana numeru "revision" wymusza ponowne pobranie danego pliku.
precaching.precacheAndRoute([
  { url: './', revision: '1' },
  { url: 'index.html', revision: '1' },
  { url: 'index.js', revision: '1' },
  { url: 'manifest.json', revision: '1' },
  { url: 'offline.html', revision: '1' },
]);

// 2. Strony HTML: najpierw sieć, a gdy jej nie ma - wersja z cache.
routing.registerRoute(
  ({ request }) => request.mode === 'navigate',
  new strategies.NetworkFirst({ cacheName: 'strony' })
);

// 3. Obrazy: najpierw cache, sieć tylko gdy obrazka jeszcze nie mamy.
routing.registerRoute(
  ({ request }) => request.destination === 'image',
  new strategies.CacheFirst({ cacheName: 'obrazy' })
);

// 4. Gdy nie ma ani sieci, ani strony w cache - pokazujemy własną stronę offline.
routing.setCatchHandler(async ({ request }) => {
  if (request.mode === 'navigate') {
```

```
    return precaching.matchPrecache('offline.html');
  }
  return Response.error();
});
```

W pierwszej linijce importujemy bibliotekę **Workbox**, która upraszcza pisanie service workerów. Następnie:

1. za pomocą modułu `precaching` zapisujemy w pamięci podręcznej podstawowe pliki aplikacji już w momencie instalacji service workera — dzięki temu aplikacja uruchomi się offline nawet przy pierwszym powrocie bez sieci,
2. korzystając z modułu ścieżkowania (`routing`) rejestrujemy ścieżki dla stron HTML i dla wszystkich obrazów, decydując o sposobie buforowania,
3. na koniec ustawiamy obsługę sytuacji, w której żądania nie da się obsłużyć — wtedy wyświetlamy stronę `offline.html`.

Najczęściej używane strategie buforowania:

- **CacheFirst()** - PWA spróbuje odczytać dane z pamięci podręcznej przed połączeniem się z Internetem. Stosujemy, gdy pliki nie zmieniają się zbyt często, np. obrazy,
- **NetworkFirst()** - PWA połączy się najpierw z Internetem. Jeżeli jesteśmy w trybie offline, odczyta dane z pamięci podręcznej. Stosujemy, gdy pliki często się zmieniają,
- **StaleWhileRevalidate()** - PWA od razu zwraca dane z pamięci podręcznej, a w tle pobiera ich nowszą wersję na następny raz. Kompromis między szybkością a aktualnością (np. pliki CSS/JS).

Tworzymy jeszcze plik `offline.html`:

```
<!DOCTYPE html>
<html lang="pl">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Brak połączenia</title>
  </head>
  <body>
    <h1>Jesteś offline</h1>
    <p>Ta strona nie jest dostępna bez połączenia z Internetem. Spróbuj ponownie później.
</p>
  </body>
</html>
```

## Dla zainteresowanych: service worker bez Workboxa

Workbox ukrywa szczegóły — warto wiedzieć, co robi „pod spodem". Ten sam efekt (pliki szkieletu w cache, strony: najpierw sieć, obrazy i pozostałe pliki: najpierw cache, strona offline) można uzyskać samym API przeglądarki: zdarzeniami `install`, `activate`, `fetch` oraz obiektem `caches`. Poniższy kod **zastępuje** całą zawartość `service-worker.js` (w dalszej części laboratorium korzystamy z wersji z Workboxem):

```
// Zmiana nazwy cache (np. na 'pwa-v2') = nowa wersja aplikacji; stare cache usuwamy w
"activate".
const CACHE = 'pwa-v1';
const PLIKI = ['./', 'index.js', 'manifest.json', 'offline.html', 'android.png'];

// 1. Instalacja: zapisujemy pliki szkieletu w cache.
self.addEventListener('install', (event) => {
  event.waitUntil(caches.open(CACHE).then((cache) => cache.addAll(PLIKI)));
});

// 2. Aktywacja: usuwamy cache z poprzednich wersji.
self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((nazwy) =>
      Promise.all(nazwy.filter((n) => n !== CACHE).map((n) => caches.delete(n)))
    )
  );
});

// 3. Każde żądanie strony przechodzi przez service workera.
self.addEventListener('fetch', (event) => {
  const { request } = event;
  if (request.method !== 'GET') return;

  if (request.mode === 'navigate') {
    // Strony HTML: najpierw sieć (i kopia do cache), bez sieci - cache, a na końcu
offline.html.
    event.respondWith(
      fetch(request)
        .then((odp) => {
          const kopia = odp.clone();
          caches.open(CACHE).then((cache) => cache.put(request, kopia));
          return odp;
        })
        .catch(async () => {
          const zCache = await caches.match(request);
          if (zCache) return zCache;
          const offline = await caches.match('offline.html');
          // Nowa odpowiedź z tą samą treścią - serwer mógł przekierować offline.html ->
offline,
          // a przeglądarka nie przyjmie "przekierowanej" odpowiedzi jako strony.
          return new Response(offline.body, { headers: offline.headers });
        })
    );
    return;
  }

  // Pozostałe pliki (obrazy, skrypty, manifest): najpierw cache, a gdy brak - sieć (i kopia
do cache).
  event.respondWith(
    caches.match(request).then(
      (zCache) =>
        zCache ||
        fetch(request).then((odp) => {
          const kopia = odp.clone();
```

```
            caches.open(CACHE).then((cache) => cache.put(request, kopia));
            return odp;
          })
      )
    );
  });
```

Porównując obie wersje, widzimy, co daje nam Workbox: strategie (`NetworkFirst`, `CacheFirst`) i wersjonowanie plików (`revision`) zamiast ręcznie pisanych `fetch` / `caches.match` i zmiany nazwy cache.

## Cykl życia service workera — ważne podczas pracy

Service worker po zainstalowaniu **działa dalej, nawet gdy zmienimy pliki na dysku**. Nowa wersja `service-worker.js` jest instalowana w tle i czeka, aż zamkniemy wszystkie karty z aplikacją. W praktyce oznacza to, że po edycji plików często zobaczymy w przeglądarce **starą wersję**. Podczas pracy nad labem:

- w DevTools → **Application** → **Service workers** zaznaczamy opcję **Update on reload**,
- w razie problemów w **Application** → **Storage** klikamy **Clear site data**,
- po zmianie plików z listy `precacheAndRoute` (np. `index.html`, `index.js`) **zwiększamy ich** `revision` (np. z `'1'` na `'2'`) i zapisujemy `service-worker.js`. Pliki z tej listy są podawane prosto z cache — bez zmiany `revision` nie pomoże ani odświeżenie strony, ani **Update on reload**.

Do tej sekcji wracamy za każdym razem, gdy po zmianie plików przeglądarka pokazuje starą wersję.

## Uruchomienie aplikacji

Gdy zapiszemy wszystkie pliki, możemy uruchomić lokalny serwer w folderze projektu:

```
npx serve
```

Przy pierwszym uruchomieniu `npx` zapyta o zgodę na pobranie pakietu `serve` — potwierdzamy. W terminalu pojawi się adres aplikacji (np. `http://localhost:3000`), który zostanie też skopiowany do schowka. Uruchamiamy przeglądarkę Google Chrome i wklejamy ten adres.

Nie potrzebujemy tutaj HTTPS — przeglądarki traktują adres `localhost` jako bezpieczny kontekst, więc service worker i instalacja działają lokalnie bez certyfikatu.

Przy pierwszym wejściu na stronę service worker dopiero się instaluje i jeszcze jej nie obsługuje (np. obrazek zostaje pobrany z pominięciem service workera). Dlatego **odświeżamy stronę (F5)**, a dopiero potem otwieramy narzędzia deweloperskie (prawy przycisk myszy → **Zbadaj**, lub klawisz **F12**). Przechodzimy do zakładki **Application** i sprawdzamy:

1. **Manifest** - powinny być widoczne nazwa, kolory i lista ikon. W sekcji **Installability** (oraz w ostrzeżeniach na górze) przeglądarka wypisze, czego brakuje, aby aplikację dało się zainstalować. Nie powinno tam być żadnych błędów. Ostrzeżenia *Richer PWA Install UI won't be available … Please add at least one screenshot …* możemy pominąć — dotyczą opcjonalnych zrzutów ekranu (pole `screenshots` w manifeście), wyświetlanych w bogatszym oknie instalacji, i nie blokują instalacji,

2. **Service workers** - service worker powinien mieć status **activated**. Obok zobaczymy **is running** albo **is stopped** — oba są poprawne: Chrome usypia nieużywanego service workera po kilkudziesięciu sekundach i budzi go przy kolejnym żądaniu,
3. **Cache storage** - powinny być widoczne cache utworzone przez Workbox: `workbox-precache-...` (pliki z listy precache) oraz `obrazy`. Cache `strony` pojawi się dopiero wtedy, gdy odwiedzimy stronę HTML spoza listy precache.

Następnie testujemy tryb offline: w **Application** → **Service workers** zaznaczamy **Offline** (albo po prostu zatrzymujemy serwer w terminalu klawiszami **Ctrl+C**) i odświeżamy stronę. Aplikacja powinna nadal się wyświetlić — razem z obrazkiem. Gdy w pasku adresu wpiszemy nieistniejącą podstronę (np. `/abc`), powinna pojawić się nasza strona `offline.html`. Po teście odznaczamy **Offline** albo — jeśli zatrzymaliśmy serwer — uruchamiamy go ponownie poleceniem `npx serve`.

Możemy jeszcze wygenerować raport **Lighthouse**, aby sprawdzić wydajność, dostępność i dobre praktyki. Lighthouse jest wbudowany w narzędzia deweloperskie: w DevTools (**F12**) wybieramy zakładkę **Lighthouse** (jeśli jej nie widać na pasku zakładek, znajdziemy ją pod przyciskiem **»**), wybieramy urządzenie (**Mobile** lub **Desktop**) i klikamy **Analyze page load**. Najlepiej uruchamiać go w oknie **incognito** (Ctrl+Shift+N) — rozszerzenia przeglądarki zaniżają wyniki. Uwaga: od wersji 12 (2024 r.) Lighthouse **nie ma już osobnej kategorii PWA** ani „odznaki PWA" — instalowalność sprawdzamy w zakładce **Application** → **Manifest**.

Jeżeli wszystko jest poprawne, w pasku adresu Chrome pojawi się ikona **instalacji aplikacji** — możemy ją kliknąć i zainstalować aplikację na komputerze.

# Przycisk instalacji aplikacji

W Chrome i przeglądarkach na nim opartych (Edge, Opera, Samsung Internet) możemy wyświetlić własny przycisk instalacji.

**Najpierw odinstalowujemy aplikację**, jeśli zainstalowaliśmy ją w poprzednim kroku: w oknie zainstalowanej aplikacji otwieramy menu ⋮ → **Odinstaluj „Moja pierwsza PWA"** (albo w Chrome wchodzimy na `chrome://apps`, klikamy ikonę prawym przyciskiem → **Usuń z Chrome**). Przeglądarka proponuje instalację — i pokaże nasz przycisk — **tylko wtedy, gdy aplikacja nie jest zainstalowana**.

Na stronie głównej, wewnątrz `<body>`, tworzymy przycisk:

```
<button class="install-button" hidden>Zainstaluj aplikację</button>
```

Następnie w pliku `index.js` (pod kodem rejestrującym service workera) dopisujemy kod obsługujący przycisk:

```
let deferredPrompt = null;
const installBtn = document.querySelector('.install-button');

window.addEventListener('beforeinstallprompt', (e) => {
  // Blokujemy domyślny komunikat przeglądarki - pokażemy go po kliknięciu naszego
przycisku.
  e.preventDefault();
```

```
  // Zapamiętujemy zdarzenie, aby wywołać je później.
  deferredPrompt = e;
  // Pokazujemy przycisk instalacji.
  installBtn.hidden = false;
});

installBtn.addEventListener('click', async () => {
  if (!deferredPrompt) return;
  installBtn.hidden = true;
  // Wyświetlamy okno instalacji.
  deferredPrompt.prompt();
  // Czekamy na decyzję użytkownika.
  const { outcome } = await deferredPrompt.userChoice;
  console.log(outcome === 'accepted' ? 'Użytkownik zainstalował aplikację' : 'Użytkownik
anulował instalację');
  deferredPrompt = null;
});

window.addEventListener('appinstalled', () => {
  installBtn.hidden = true;
  console.log('Aplikacja została zainstalowana');
});
```

Zapisujemy pliki i odświeżamy stronę. **Jeśli przycisku nie ma** (także w DevTools → **Elements** nie widać elementu `<button>`), przeglądarka pokazuje starą wersję `index.html` i `index.js` z cache service workera — rozwiązanie jest w sekcji **„Cykl życia service workera — ważne podczas pracy"** powyżej: w `service-worker.js` zwiększamy `revision` zmienionych plików (`'./'`, `index.html`, `index.js`), zaznaczamy **Update on reload** i odświeżamy stronę. Jeśli `<button>` jest w kodzie strony, ale pozostaje ukryty — aplikacja jest nadal zainstalowana (patrz wyżej: najpierw odinstalowujemy).

W konsoli pojawi się komunikat *Banner not shown: beforeinstallpromptevent. preventDefault() called ...* — to nie błąd, tylko informacja, że sami przejęliśmy okno instalacji.

Zdarzenie `beforeinstallprompt` jest dostępne **tylko w przeglądarkach opartych na Chromium**. W Safari (iOS, macOS) i w Firefoksie przycisk się nie pojawi — i dobrze, bo aplikacja nadal działa (progresywne ulepszanie). Na **iPhonie** aplikację instalujemy ręcznie: w Safari przycisk **Udostępnij** → **Do ekranu początkowego**.

# Testowanie na telefonie z Androidem

Aplikację uruchomioną lokalnie możemy otworzyć na telefonie z Androidem bez HTTPS — przekierowując port z komputera na telefon przez kabel USB. Na telefonie musimy mieć włączone **Opcje programisty** i **Debugowanie USB**. Podłączamy telefon kablem i w przeglądarce Chrome na komputerze wpisujemy:

```
chrome://inspect/#devices
```

a następnie zaznaczamy **Discover USB devices** (i akceptujemy na telefonie prośbę o zezwolenie na debugowanie). Klikamy **Port forwarding** i wpisujemy port, który wyświetlił się nam podczas uruchamiania serwera, np. `3000` oraz `localhost:3000`.

U dołu zaznaczamy **Enable port forwarding** i klikamy przycisk **Done**. W polu **Open tab with url** wpisujemy `localhost:3000` i klikamy przycisk **Open**. Na telefonie w Chrome instalujemy aplikację (nasz przycisk lub menu → **Zainstaluj aplikację** / **Dodaj do ekranu głównego**). Uwaga! Ten sposób działa tylko na Androidzie.

# Publikacja z szyfrowanym połączeniem (HTTPS)

Poza `localhost` aplikacja PWA **musi być serwowana przez HTTPS** — inaczej service worker się nie zarejestruje, a aplikacji nie da się zainstalować. Najprościej skorzystać z darmowego hostingu, który zapewnia HTTPS automatycznie, np. **Firebase Hosting**, **GitHub Pages**, Netlify czy Cloudflare Pages. Skorzystamy z Firebase.

1. Wchodzimy na https://console.firebase.google.com i tworzymy nowy projekt (**Create a new Firebase project**). Podajemy nazwę projektu; asystenta AI (**Gemini**) ani Google Analytics nie potrzebujemy — możemy je wyłączyć.

2. Sprawdzamy, czy mamy Firebase CLI. W pracowni jest już zainstalowane:

   ```
   firebase --version
   ```

   Praca w domu: jeśli polecenie nie jest rozpoznawane, instalujemy Firebase CLI (wymaga Node.js w wersji 20 lub nowszej):

   ```
   npm install -g firebase-tools
   ```

3. Logujemy się (otworzy się przeglądarka, w której wybieramy konto Google):

   ```
   firebase login
   ```

   Przed otwarciem przeglądarki CLI zada dwa pytania — **Enable Gemini in Firebase features?** oraz **Allow Firebase to collect CLI and Emulator Suite usage and error reporting information?** — na oba możemy odpowiedzieć `N`. Po zalogowaniu w terminalu pojawi się **Success! Logged in as ...**.

   > ⚠️ **W pracowni: na koniec zajęć koniecznie się wylogowujemy!**
   >
   > ```
   > firebase logout
   > ```
   >
   > Logowanie zapisuje dostęp do Twojego konta Google na komputerze. Bez wylogowania kolejna osoba przy tym stanowisku może publikować na Twoje projekty Firebase.

4. W folderze `lab_01` (czyli **nadrzędnym** względem `public` z naszą aplikacją) inicjalizujemy projekt:

   ```
   firebase init hosting
   ```

   Odpowiadamy na pytania:

   - **Are you ready to proceed?** `Y`

- **Please select an option:** `Use an existing project` → wybieramy projekt utworzony w kroku 1,
- **What do you want to use as your public directory?** `public` (nazwa folderu z naszą aplikacją),
- **Configure as a single-page app (rewrite all urls to /index.html)?** `N`
- **Set up automatic builds and deploys with GitHub?** `N`
- **File public/index.html already exists. Overwrite?** `N` — **nie nadpisujemy** naszego pliku!
- **Would you like to install agent skills for Firebase?** `N`

Firebase utworzy pliki `firebase.json`, `.firebaserc` i `.gitignore` oraz (jeśli go nie było) `public/404.html`.

5. Publikujemy aplikację:

```
firebase deploy
```

6. Sprawdzamy, czy aplikacja działa, otwierając adres podany w linijce **Hosting URL**, np.:

```
https://xyz.web.app
```

Teraz możemy bezpiecznie połączyć się z naszą stroną z dowolnego urządzenia i zainstalować aplikację na telefonie (Android: przycisk instalacji lub menu Chrome; iPhone: Safari → Udostępnij → Do ekranu początkowego).

Uwaga: gdy jesteśmy online, nieistniejąca podstrona (np. `/abc`) na Firebase pokaże plik `404.html` utworzony przez `firebase init` — nasza strona `offline.html` pojawi się dopiero bez połączenia. Po każdej zmianie plików aplikacji pamiętamy o zwiększeniu `revision` w `service-worker.js` (patrz „Cykl życia service workera") i ponownym `firebase deploy`.

> ⚠️ **Koniec zajęć w pracowni:** wylogowujemy się z Firebase CLI poleceniem `firebase logout`.

# Zadanie

Na podstawie powyższej instrukcji przygotuj aplikację PWA, która spełnia poniższe warunki (każdy z nich sprawdź w DevTools):

1. W zakładce **Application** → **Manifest** nie ma błędów (ostrzeżenia o zrzutach ekranu — *Richer PWA Install UI* — można pominąć), a aplikację da się zainstalować (w Chrome na komputerze pojawia się ikona instalacji w pasku adresu).
2. Manifest zawiera nazwę, kolory i ikony co najmniej 192×192 i 512×512 px.
3. Service worker jest aktywny (status **activated**).
4. Po włączeniu trybu **Offline** i odświeżeniu strony aplikacja nadal się wyświetla, a nieznana podstrona pokazuje własną stronę `offline.html`.
5. Własny przycisk instalacji działa w Chrome i jest ukryty w przeglądarkach, które go nie obsługują.
6. Aplikacja jest opublikowana przez HTTPS (Firebase Hosting lub GitHub Pages) i zainstalowana na telefonie.
7. Raport **Lighthouse** daje wynik **100** w kategoriach **Accessibility** i **Best Practices** — popraw aplikację zgodnie z jego wskazówkami (pozycje z sekcji *Trust and Safety* nie wpływają na wynik i dotyczą konfiguracji serwera — nie trzeba ich poprawiać).

Kod umieść w repozytorium GitHub w folderze `lab_01/`, a w pliku `README.md` podaj adres opublikowanej aplikacji oraz zrzut ekranu zainstalowanej aplikacji na telefonie.

Źródła:

- https://web.dev/learn/pwa
- https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable
- https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/How_to/Trigger_install_prompt
- https://web.dev/learn/pwa/caching/
- https://developer.chrome.com/docs/workbox
- https://developer.chrome.com/docs/devtools/progressive-web-apps
- https://web.dev/articles/maskable-icon
- https://developer.chrome.com/docs/devtools/remote-debugging/
- https://firebase.google.com/docs/hosting/quickstart
- https://www.freecodecamp.org/news/build-a-pwa-from-scratch-with-html-css-and-javascript/